基于 Gamma 分布回归、风险优化与堆叠式集成模型的综合分析：NIPT 时点选择及胎儿异常判定研究

摘要

产前检测是预防出生缺陷、保障母婴健康的关键环节。在各类产前检测手段中，NIPT（Non-invasive Prenatal Test，即无创产前检测）已成为一项重要技术。本文研究各类孕妇群体适宜的 NIPT 时点，并分析检测的准确性，旨在保障婴儿和孕妇的身心健康。

针对问题一，为了分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相关特性，进行了**斯皮尔曼相关系数**分析，发现 Y 染色体浓度与 BMI 弱负相关，相关系数-0.130，与孕周数弱正相关，相关系数 0.042，难以构建经典回归模型。为了给出最佳关系模型，通过 **AIC/BIC 准则**选择 Gamma 分布描述 Y 染色体浓度分布，对 Gamma 分布形状参数，尺度参数和自变量 BMI，孕周数进行**对数线性回归**作为链接函数，通过**极大似然估计**进行参数估计，最终构建了基于对数线性回归的 Gamma 分布族累积概率函数模型$F(x; bmi, 孕周数)$，BMI、孕周数对形状参数和尺度参数的 **p 值均小于 0.05**，通过显著性检验。

针对问题二，为了根据 BMI 合理分组并确定最佳 NIPT 时间，先将 BMI 均分为 30 组，每组构建了以提高 NIPT 成功率和降低治疗窗口期风险为目标，BMI，孕周数取值范围为约束，NIPT 时间为决策变量的**优化模型**，通过黄金分割搜索与抛物线插值优化算法求解，再依据每组最佳 NIPT 时间 **K-means 聚类**分组，最终分为 3 类，低 BMI（20-26.6），中 BMI（27.16-33.78），高 BMI（33.78-47），分别对应最佳检测时间 14.75 周，23.59 周，29.93 周，高 BMI 孕妇可早去并多次检查，提高检测可靠度降低风险。分析检测误差，Y 染色体浓度和 BMI 误差可能导致假阴性或假阳性，增大治疗风险。

针对问题三，为了更综合考虑各种因素对最佳 NIPT 的时间影响，本文在问题二基础上增加形状参数约束**控制检测误差**，增加检测成功率约束保证 Y 染色体浓度的达标比例，构建出优化模型，SLSQP 算法求解后，绘制肘部图确定分 4 类，使用 K-means 聚类最终分为极低 BMI 组（19-26.2），低 BMI 组（26.29-38.36），高 BMI 组（38.36-40），极高 BMI 组（40-47），分别对应最佳检测时间 10.73 周，15.40 周，22.67 周，30 周，极高 BMI 类孕妇可早去并多次检查降低风险。分析检测误差，Y 染色体浓度和身高、体重、生日等因素的测量误差可能导致假阴性或假阳性，增大治疗风险。

针对问题四，对于这种分类预测问题，鉴于其中异常样本数据过少导致数据失衡，单个机器学习模型效果不佳，故我们使用多个机器学习模型作为基模型，同时通过对各代表性指标进行**检验**以及**交互**、**过采样**等方法弥补原始数据的不足，同时通过**网格搜索**优化模型参数来提升解释力，最终将基模型得到的新数据放入元模型中，不仅使各基模型互补，预测结果准确率、召回率、精确率分别达到 **88.62%，89.36%，87.50%**，使得女胎异常判定更具权威。

关键词：NIPT Gamma 分布 K-means 聚类 优化模型 堆叠式集成模型

一、 问题重述

1.1 问题背景

NIPT 是一项重要的产前检测技术：它通过采集孕妇外周血，提取并检测其中胎儿的游离 DNA 片段，进而分析胎儿染色体是否存在异常。目前，该技术凭借无创性、可早期检测、准确性高等突出优势，已成为临床领域筛查胎儿染色体异常的核心手段之一。根据国家卫生健康委员会 2022 年修订的《产前诊断技术管理办法》，NIPT 已被明确列为唐氏综合征、爱德华氏综合征、帕陶氏综合征的一线筛查手段，这三种综合征的发生，分别由胎儿 21 号、18 号、13 号染色体的游离 DNA 片段比例（简称“染色体浓度”）是否异常决定，其结果准确性直接决定了胎儿健康判定与后续干预方案的制定[1]。

1.2 问题提出

临床实践中，NIPT 的准确性判定主要依据胎儿性染色体（男性胎儿染色体为 XY，女性胎儿染色体为 XX）的浓度情况。一般来说，孕妇处于怀孕 10 周至 25 周期间时，能够对胎儿性染色体的浓度进行检测。若男性胎儿的 Y 染色体浓度达到或超过 4%，女性胎儿的 X 染色体浓度不存在异常，那么便可以认定 NIPT 的检测结果基本准确，反之，则难以保证其结果符合准确性要求。

同时，检测时点与母婴风险紧密挂钩——《中华围产医学杂志》2023 年发表的多中心研究显示，12 周以内（早期）检测可使胎儿异常干预的窗口期充足，风险最低；13-27 周（中期）检测会因干预时间缩短导致风险升高；28 周以后（晚期）检测则可能错过有效治疗时机，风险极高[2]。

实践证明，男胎 Y 染色体浓度与孕妇孕周数、BMI 密切相关，孕妇个体差异大，简单经验分组和统一检测时点会影响 NIPT 准确性。依据 BMI 对孕妇进行合理分组（例如：[20,28)，[28,32)，[32,36)，[36,40)，40 以上），分别确定最佳时点可降低潜在风险。附件提供了某地区（多为高 BMI）孕妇的 NIPT 数据，实际检测存在测序失败及多次检测以提高结果可靠性的情况。需据此建立数学模型，解决如下问题：

(1) 分析胎儿 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关特性，建立相应关系模型并检验其显著性。

(2) 鉴于男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度最早达标时间（浓度≥4%的最早时间）的主要因素，对男胎孕妇的 BMI 合理分组，给出每组的 BMI 区间和最佳 NIPT 时点以最小化孕妇潜在风险，同时分析检测误差对结果的影响。

(3) 综合身高、体重、年龄等因素、检测误差及胎儿 Y 染色体浓度达标比例（浓度≥4%的比例），根据男胎孕妇的 BMI 合理分组，给出每组的最佳 NIPT 时点以最小化孕妇潜在风险，同时分析检测误差对结果的影响。

(4) 由于孕妇和女胎均不携带 Y 染色体，需判定女胎是否异常。以女胎孕妇的 21 号、18 号和 13 号染色体非整倍体（AB 列）为判定结果，综合 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，给出女胎异常的判定方法。

二、 问题分析

2.1 问题一的分析

问题一中，先对年龄、身高、体重、检测孕周、孕妇 BMI、Y 染色体浓度六要素指标进行斯皮尔曼相关性分析，并绘制热力图，发现相关系数过小；随后对检测孕周、BMI、Y 染色体浓度进行统计量的计算，绘制直方图后观察趋势，发现分布过于杂乱，三者关系难以进行回归或拟合。结合附件中数据存在高 BMI 数量多、重复测量、测序存在误差等特征，可以选取适配非正态数据、稳健性和可解释性强的概率分布模型解决回归模型拟合不佳的问题。首先，确定因变量和核心自变量，明确核心自变量的选择理由，拟合其概率分布并关联自变量对分布参数的影响，从而实现相关特性分析。最终选择卡方检验，显著性水平通常设定为 0.05，p 值小于 0.05 则表示变量之间存在显著性差异。

2.2 问题二的分析

问题二中，核心需求是对男胎孕妇 BMI 进行合理分组并确定 NIPT 检测的最佳孕周时点，同时需要分析检测误差对结果的影响。其本质是基于问题一得到的 Y 染色体浓度条件分布函数，建立优化模型，找到每组 BMI 下 Y 浓度≥4%概率最高的孕周。因此，可在问题一的基础上建立目标函数，目标是使其最大化。约束条件既需要考虑参数的取值范围，又需要兼顾风险窗口期。然后对 BMI 进行分组，在每个分组中找到使目标函数最大的最优 t。

为了根据 BMI 合理分组并确定最佳 NIPT 时间，先将 BMI 均分为若干组，每组构建了以提高 NIPT 成功率和降低治疗窗口期风险为目标的优化模型，通过黄金分割搜索与抛物线插值优化算法求解，再依据每组最佳 NIPT 时间进行 K-means 聚类分组，进一步求解并分析检测误差。

2.3 问题三的分析

问题三中，为了更综合地考虑各种因素对最佳 NIPT 的时间影响，考虑基于问题二，增加形状参数约束表示对检测误差的控制，增加检测成功率约束以保证 Y 染色体浓度的达标比例，并考虑年龄越大 Y 染色体浓度达标时间越长，构建出优化模型，利用 SLSQP 算法求解，绘制肘部图确定分类，再使用 K-means 聚类确定最终分组，求解并分析检测误差。

2.4 问题四的分析

问题四中，因单个机器学习分类模型效果均不佳，故考虑训练逻辑回归、决策树和 SVM 三个基模型，扩大网格搜索的参数范围，以找到可能存在的更优参数组合，采用过采样平衡数据，然后将它们对训练集和测试集的预测结果作为新的特征，用一个元模型进行最终的预测。

三、 模型假设

1. 问题一模型求解之前，假设 Y 染色体浓度服从 Gamma 分布。
2. 问题二、问题三中，假设 NIPT 可以和 Y 染色体浓度一起检测。

四、 符号说明

符号	说明	单位
t	孕周数	周
bmi	BMI	kg/m^2
x	Y 染色体浓度	%
α	形状参数	/
β	尺度参数	/
ACC	准确率	/
TPR	召回率	/
precision	精确度	/

五、 数据预处理

在数据分析、模型建立与求解的过程中，数据预处理是一个不可或缺的步骤。它能够提高数据的质量和可用性，为后续的问题求解奠定坚实基础。针对附件提供的数据集，预处理操作如下：

(1) **筛选对读段数。**杨呈勇、甘海燕在《一种检测胎儿染色体非整倍体的方法及系统》（专利号：CN105483229B，2018 年 10 月 16 日授权）中提到，仅需 3000000 条唯一比对的读段，就能检出胎儿浓度 4%以上的染色体非整倍体，具备成本相对较低且灵敏度高的优势[3]。题目中系统的组成模块与该发明近乎一致，且该发明已获授权，所涉及的检测胎儿染色体非整倍体的方法及系统具有临床应用性和推广性。因此，为保证数据的准确性和可靠性，首先筛选出唯一比对读段数在 3000000 以下的数据，并将其筛除，以排除可能存在的低质量数据，减少对后续分析和求解的干扰。

(2) **GC 含量控制。**GC 含量是测序数据质量评估的重要指标之一。正常 GC 含量范围是 40%～60%，GC 含量过高、过低或分布异常都可能意味着测序质量存在问题。因此，在数据预处理时，要求 GC 含量控制在 0.395 到 0.6 的范围内，这既贴合正常生理范围，又可通过微小的下界浮动兼顾数据波动，确保数据的一致性和可比性。

(3) **妊娠方式编码。**针对三类妊娠方式，分别进行名义变量编码（仅用于区分类别，数值大小无实际意义，不具有内在等级）。“自然妊娠”“人工授精”“试管婴儿”的编码分别为 1、2、3，转化为离散型变量，便于输入数值型数据进行相关性分析以及利用统计软件进行数据存储和运算。

(4) **孕周数标准化。**将孕周数由“周数+天数”统一化为以周为单位的七进制数据，例如检测孕周 11w+6 化为 11.8571428571429，便于进行标准化分析。

(5) **怀孕次数统一。**对于怀孕次数≥3 次的数据，统一按 3 次计算，为模型的建立与求解提供便利。

(6) **针对表 1 男胎监测数据，去除离群数据。**依据 Y 染色体浓度，识别并去除离群数据，有助于消除异常值对数据分析的影响，提高结果的准确性。

(7) **针对表 2 女胎检测数据，处理染色体异常数据。**AB 列（染色体的非整倍体）用于指示 13 号、18 号、21 号染色体的数量情况：当显示为“非整倍体”时，说明这些染色体存在数量异常；当显示为空白时，则意味着这些染色体的数量正常。提取

AB 列内容，并将其拆分为三列：“染色体异常 13”“染色体异常 17”“染色体异常 21”，分别进行编码，“无异常”和“有异常”编码分别为 0 和 1，便于后续的分析与求解。



图 1 数据预处理流程图

六、 模型的建立与求解

6.1 问题一模型的建立与求解

6.1.1 模型的建立

对于问题一，需要分析胎儿 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关特性，并建立相应关系模型，检验其显著性。因此，先绘制热力图，对年龄、身高、体重、检测孕周、孕妇 BMI、Y 染色体浓度六个要素指标进行斯皮尔曼相关性分析，热力图中的每个单元格的颜色和数值反映所对应的两个变量的斯皮尔曼相关系数。

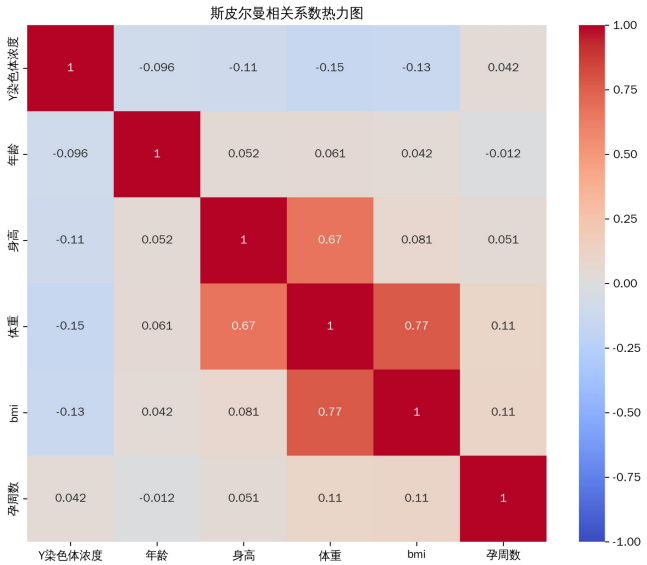


图 2 斯皮尔曼相关系数热力图

由热力图可看出，Y 染色体浓度与年龄、身高、体重、BMI、孕周数的相关系数均较低，表明 Y 染色体浓度与这些变量的相关性较弱。

随后，对检测孕周数、BMI、Y 染色体浓度这三个核心指标，逐一开展统计量的计算，绘制直方图，观察三个指标各自的分布形态。

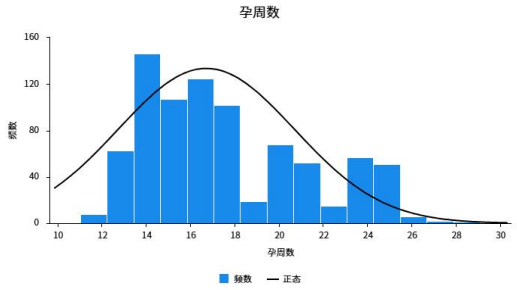


图 3 孕周数数据分布直方图

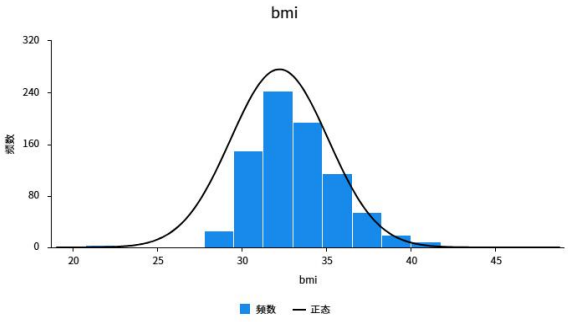


图 4 BMI 数据分布直方图

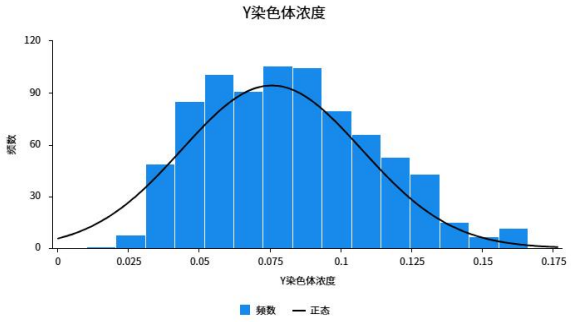


图 5 Y 染色体浓度数据分布热力图

然而，从绘制的直方图结果来看，数据分布呈现出明显的杂乱无章特征。具体而言，检测孕周数的直方图未呈现出预期的正态分布或特定规律的偏态分布，无明显的峰值集中区域，也未呈现出稳定递增或递减趋势；BMI 的分布较为杂乱，不符合正态分布；Y 染色体浓度的直方图也呈现出碎片化分布的特征。这种杂乱的分布状态无法满足回归或拟合分析对数据分布的基本假设，直接导致三者之间的关系难以进行回归或拟合。

考虑到附件所给的数据中高 BMI 数量多，且存在重复测量、测序误差等特征，结合上述分析结果，可以选取适配非正态数据的、稳健性和灵活性较强的概率分布模型解决三者关系难以进行回归或拟合的问题。

- **明确建模对象，定义核心变量**

使用概率分布建立模型，分析胎儿 Y 染色体浓度与孕周、BMI 的相关特性。核心变量需限定为：

- 自变量：孕周数（连续变量）、BMI（连续变量）。
- 因变量：胎儿 Y 染色体浓度。

- **构建概率分布模型**

第一步，先不考虑自变量与因变量之间的关系，独立拟合胎儿 Y 染色体浓度的边际分布，确定其分布规律，为后续关联自变量奠定基础。

Gamma 分布是一种连续概率分布，用于描述实值随机变量，适用于非负连续变量，偏态分布。其形状参数控制偏度，尺度参数控制集中趋势。本题目中，胎儿 Y 染色体浓度≥0，且高 BMI 孕妇浓度可能偏低、波动大，符合 Gamma 分布的偏态特性。因此，绘制胎儿 Y 染色体浓度的直方图与估计的 Gamma 分布曲线对比图，可视化如下。

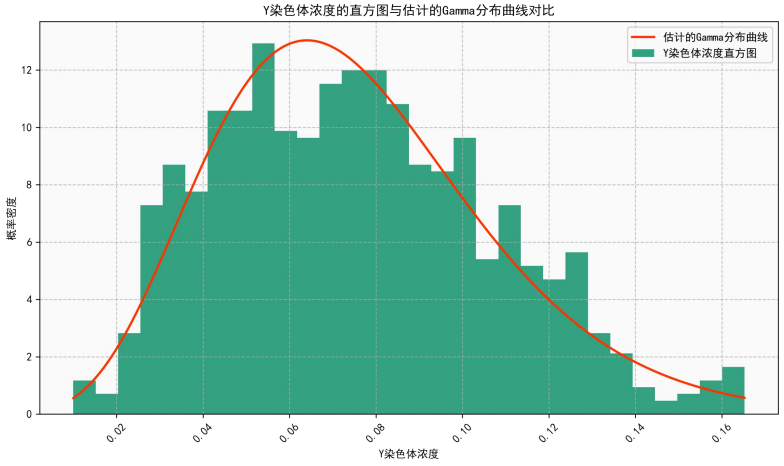


图 6　Y 染色体浓度的直方图与估计的 Gamma 分布曲线对比图

Gamma 分布的概率密度函数为：

$$f(y;\alpha,\beta)=\frac{1}{\Gamma(\alpha)\beta^{\alpha}}y^{\alpha-1}e^{-y/\beta} \qquad (1.1)$$

（α 为形状参数，β 为尺度参数）

下一步，关联自变量。将自变量与分布参数关联，构建条件概率分布，即给定孕周（x_1）、BMI（x_2）时，Y 浓度的分布参数是x_1、x_2的函数。这是概率分布模型回应问题一，分析 Y 染色体浓度与孕周数、BMI 的相关特性的关键环节。

将α、β 建模为孕周（x_1）、BMI（x_2）的函数，使用对数线性形式，确保参数为正，避免参数估计出现负值：

$$\ln(\alpha)=\beta_0+\beta_1x_1+\beta_2x_2 \text{（形状参数与孕周、BMI 的对数线性关系）} \qquad (1.2)$$

$$\ln(\beta)=\gamma_0+\gamma_1x_1+\gamma_2x_2 \text{（尺度参数与孕周、BMI 的对数线性关系）} \qquad (1.3)$$

其中：

β_0、γ_0为常数项；

β_1、γ_1为孕周数、BMI 对形状参数α的关联系数；

β_2、γ_2为孕周数、BMI 对尺度参数β的关联系数。

6.1.2 模型的求解

关联自变量的系数通过极大似然估计求解，其本质是找到一组系数，使观测到的 Y 染色体浓度数据在给定孕周数、BMI 下的“条件概率”最大。具体步骤如下：

第一步，构建条件似然函数。

设共有 n 男胎样本，第 i 个样本的观测值为：

- y_i：第个样本的 Y 染色体浓度；

- x_{1i}：第个样本的孕周；
- x_{2i}：第个样本的 BMI。

若 Y 染色体浓度服从 Gamma 分布，则第 i 个样本的条件概率密度函数为：

由于样本独立，整体条件似然函数为所有样本的乘积：

$$f(y_i|x_{1i},x_{2i};\alpha_i,\beta_i)=\frac{1}{\Gamma(\alpha_i)\beta_i^{\alpha_i}}y_i^{\alpha_i-1}e^{-y_i/\beta_i} \tag{1.4}$$

其中，由对数链接函数转换可得：$\alpha_i=e^{\beta_0+\beta_1x_{1i}+\beta_2x_{2i}}$，　$\beta_i=e^{\gamma_0+\gamma_1x_{1i}+\gamma_2x_{2i}}$

由于样本之间彼此独立，整体条件似然函数为所有样本条件概率密度函数的乘积：

$$L(\beta_0,\beta_1,\beta_2,\gamma_0,\gamma_1,\gamma_2)=\prod_{i=1}^{n}f\left(y_i|x_{1i},x_{2i};\alpha_i,\beta_i\right) \tag{1.5}$$

为简化计算，取对数似然函数，将乘积转为求和，降低计算难度：

$$\ln L=\sum_{i=1}^{n}\left[(\alpha_i-1)\ln y_i-\frac{y_i}{\beta_i}-\ln\Gamma\left(\alpha_i\right)-\alpha_i\ln\beta_i\right] \tag{1.6}$$

第二步，求解对数似然函数的极值。

利用有限差分法求解以下方程组，进行系数计算，找到使ln L最大的$\beta_0,\beta_1,\beta_2,\gamma_0,\gamma_1,\gamma_2$，：

$$\begin{aligned}&\frac{\partial\ln L}{\partial\beta_0}=0,\quad\frac{\partial\ln L}{\partial\beta_1}=0,\quad\frac{\partial\ln L}{\partial\beta_2}=0\\&\frac{\partial\ln L}{\partial\gamma_0}=0,\quad\frac{\partial\ln L}{\partial\gamma_1}=0,\quad\frac{\partial\ln L}{\partial\gamma_2}=0\end{aligned} \tag{1.7}$$

第三步，得出系数估计结果，并进行系数的显著性检验。

通过计算，得到系数估计结果：

$$F(x;bmi,\text{孕周数})=\frac{\gamma\left(e^{\beta_0+\beta_1\times bmi+\beta_2\times\text{ t}},\frac{x\times e^{\beta_0+\beta_1\times bmi+\beta_2\times\text{ ttt}}}{e^{\gamma_0+\gamma_1\times bmi+\gamma_2\times\text{ t}}}\right)}{\Gamma\left(e^{\beta_0+\beta_1\times bmi+\beta_2\times\text{ 孕周数}}\right)} \tag{1.8}$$

表 1 系数估计与显著性检验结果

参数名称	估计值	标准误差	z 值	p 值
β_0	2.0619	0.0639	32.284	0.00
β_1	-0.2056	0.0119	-17.227	0.0
β_2	0.1183	0.0134	8.844	0.0
γ_0	-6.9045	0.1446	-47.734	0.0
γ_1	0.4741	0.0261	18.132	0.0
γ_2	0.2331	0.0336	6.944	0.0

在显著性检验中，当显著性水平α取0.05时，P>0.05为“不显著”，P<=0.05为“显著”。从这些参数的p值来看，所有参数p值四舍五入后均为0.0，这意味着在α取0.05的显著性水平下，这些参数都是显著的。

6.2 问题二模型的建立与求解

6.2.1 模型的建立

通过问题一的分析与求解得知：Y染色体浓度与孕周数和BM之间存在一定的统计依赖关系。因此，基于问题一中得到的 Y 染色体浓度条件分布函数，建立模型，找到每组BMI下Y染色体浓度≥4%概率最高的孕周，即：

$$P(y \geq 4\%|bmi,\mathrm{t}) = 1 - F(4\%|\mathrm{bmi},\mathrm{t}) \qquad (2.1)$$

上式表示在给定最佳NIPT时点和BMI值的情况下，Y染色体浓度达到或超过4%的概率。

- **变量及参数准备**

1. 决策变量

为制定使孕妇可能的潜在风险最小的策略，需调节每组BMI 区间上的NIPT时间节点。因为男胎的Y染色体浓度≥4%时，可以认为NIPT的结果基本准确。因此可将决策变量设置为每组BMI 区间上的NIPT时间节点，用t表示。

2. 已知参数

bmi：表示取值范围限定在20-47之间的孕妇BMI。

- **目标函数**

归一化：

$$\frac{\text{原始数据}—min}{max\ -\ min} \qquad (2.2)$$

成功率归一化：

$$\frac{1 - F(0.04; bmi, t) - (1 - 0.04274)}{1 - 0 - (1 - 0.04274)} \qquad (2.3)$$

时间风险归一化：

$$\frac{t - 10}{20} \qquad (2.4)$$

该模型目标为Y染色体浓度≥4%的概率最大，即孕妇可能的潜在风险最小。

$$\mathrm{Max}\, Z = a[1 - f(0.04|bmi, t)] - b\frac{t - 10}{20} \qquad (2.5)$$

其中：

a、b：分别为所分配的权重，$a + b = 1$

此处为医疗场景，优先考虑风险，此处a=0.2，b=0.8。

$1 - f(0.04|bmi, t)$：准确率，前面系数为正

$\frac{t-10}{20}$：风险窗口期（t 进行归一化处理），前面系数为负（因为 t 检测时间越大，风险窗口期越小）

- **约束条件**

  由附件中所给出的数据划定孕周数和 BMI 的取值范围。

  - 孕周数范围是[10-30]
  - BMI 范围是[20-47]

- **优化模型**

  综上所述，孕妇可能的潜在风险最小策略优化模型如下：

$$\text{Max } Z = a[1 - f(0.04|bmi, t)] - b\frac{t-10}{20} \tag{2.6}$$

$$s.t. = \begin{cases} t \subseteq [10,30] \\ bmi \subseteq [20,47] \end{cases} \tag{2.7}$$

6.2.2 模型的求解

利用黄金分割搜索与抛物线插值优化算法求解目标函数极值，结果如下图。

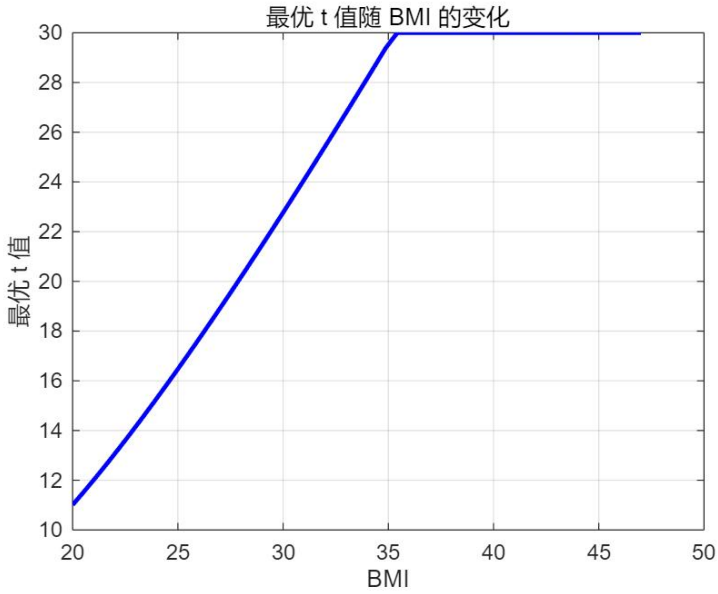


图 7 最优 t 值随 BMI 变化趋势图

表 2 问题二模型求解结果

Row	BMI（区间起点）	最优 t	最大值
样本 1	20	11.16040173	0.084248447
样本 2	20.55102041	11.70669225	0.061809106
样本 3	21.10204082	12.26949933	0.038754054
样本 4	21.65306122	12.84764201	0.015124984
样本 5	22.20408163	13.44012197	-0.009040132
……	……	……	……

样本 46	44.79591837	29.9999601	-2.951261791
样本 47	45.34693878	29.9999601	-3.126097957
样本 48	45.89795918	29.9999601	-3.295773868
样本 49	46.44897959	29.9999601	-3.459358144
样本 50	47	29.9999601	-3.616124293

6.2.3 检测误差对结果的影响

1.检验误差可能来源于：

①检验 BMI；

②检验 Y 染色体浓度。

2.对结果的影响：

①如果检验 Y 染色体浓度有误差，可能会导致检测值与真实值的偏离，这时候我们可能会误判它为假阴性或假阳性。（比如真实值是 0.04，但由于检测误差导致观测到的值是 0.038，则误判为假阴性）。

②如果检验 BMI 有误差，可能会导致它的分组错误，致使检测时间滞后或提前。若检测时间滞后，则治疗的窗口期风险增大；若检测时间提前，则浓度还未达到 4%，NIPT 正确率不高。

6.3 问题三模型的建立与求解

6.3.1 模型的建立

问题三中，男胎 Y 染色体浓度达标时间受多种因素(身高、体重、年龄等)的影响，需要综合考虑这些因素。身高和体重因素方面，由于问题一中已绘制热力图，对年龄、身高、体重、检测孕周、孕妇 BMI、Y 染色体浓度六个要素指标进行了斯皮尔曼相关性分析，可发现身高、体重与 bmi 明显强相关。年龄因素方面，年龄越大，到达 Y 染色体浓度标准的时间越长，且该结论经过了显著性检验。因此，问题三目标函数可相似于问题二，结合问题三的题目要求增添约束条件。

因此，对于问题三，可在问题二基础上增加形状参数约束表示对检测误差的控制，增加检测成功率约束保证 Y 染色体达标比例，并考虑年龄越大 Y 染色体浓度达标时间越长，建立优化模型并给出合理分组，找到每组 BMI 下 Y 染色体浓度≥4%概率最高的孕周，即：

$$P(x \geq 4\%|bmi,\mathrm{t}) = 1 - F(4\%|\mathrm{bmi},\mathrm{t}) \qquad (3.1)$$

上式表示在给定最佳 NIPT 时点和 BMI 值的情况下，Y 染色体浓度达到或超过 4%的概率。

- **变量及参数准备**

1. 决策变量

为制定使孕妇可能的潜在风险最小的策略，需调节每组 BMI 区间上的 NIPT 时间节点。因为男胎的 Y 染色体浓度≥4%时，可以认为 NIPT 的结果基本准确。因此可将决策变量设置为 Y 染色体浓度。

2. 已知参数

bmi：表示取值范围限定在 20-47 之间的孕妇 BMI。

t：表述取值范围限定在 10-30 之间的孕周数。

- **目标函数**

该模型目标为 Y 染色体浓度≥4%的概率最大，即孕妇可能的潜在风险最小。

$$\text{Max } Z = a[1 - f(0.04|bmi, t)] - b\frac{t-10}{20} \quad (3.2)$$

其中：

a、b：分别为所分配的权重，$a + b = 1$

$1 - f(0.04|bmi, t)$：准确率，前面系数为正

$\frac{t-10}{20}$：风险窗口期（t 进行归一化处理），前面系数为负（因为 t 检测时间越大，风险窗口期越小）

- **约束条件**

控制检测误差，即：

$$s.t. = \begin{cases} 10 \leq \text{孕周数} \leq 30 \\ 20 \leq bmi \leq 47 \\ 99\% \leq 1 - F(0.04) \rightarrow \text{成功率在 } 99\% \text{以上, 约束 Y 染色体浓度达标比例} \\ \alpha \geqslant 0.05 \rightarrow \text{控制离散程度} \rightarrow \text{即考虑检测误差} \end{cases} \quad (3.3)$$

- **优化模型**

综上，孕妇可能的潜在风险最小策略优化模型如下：

$$\text{Max } Z = a[1 - f(0.04|bmi, t)] - b\frac{t-10}{20} \quad (3.4)$$

$$s.t. = \begin{cases} 10 \leq \text{孕周数} \leq 30 \\ 20 \leq bmi \leq 47 \\ 99\% \leq 1 - F(0.04) \rightarrow \text{成功率在 } 99\% \text{以上，约束 Y 染色体浓度达标比例} \\ \alpha \geqslant 0.05 \rightarrow \text{控制离散程度} \rightarrow \text{即考虑检测误差} \end{cases} \quad (3.5)$$

6.3.2 模型的求解

利用 SLSQP 算法求解不同 BMI 下最优孕周数，结果如下

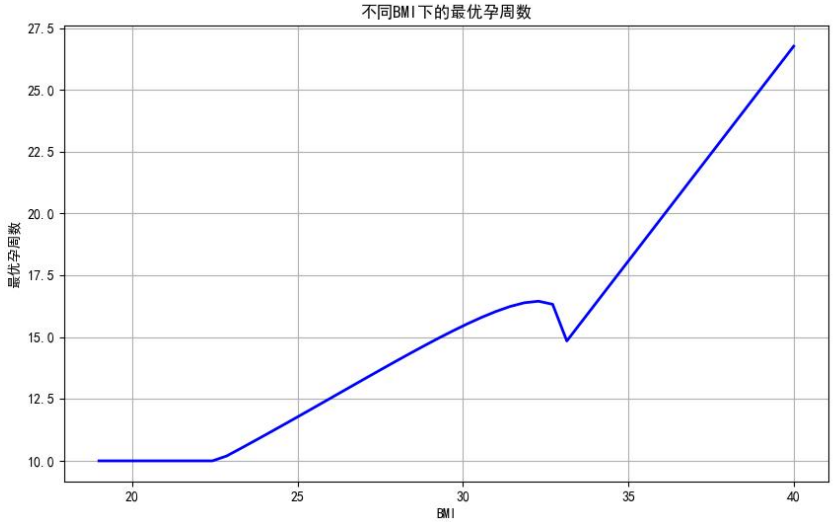


图 8 不同 BMI 下的最优孕周图

表 3 问题三模型求解结果

BMI（区间起点）	最优孕周数
19.00	10.02
19.43	10.00
19.86	10.00
20.29	10
20.71	10
……	……
38.29	23.79
38.71	24.53
39.14	25.28
39.57	26.02
40.00	26.77

接下来，利用 K-means 聚类的方法来找出每一组的最佳结果。把所有数据分成 50 个小组，即为 50 个簇，每个数据点都会归到离它最近的小组中心所在的组里。通过反复调整簇中心的位置，K-means 不断优化簇内的紧密度，从而获得尽可能紧凑的、彼此分离的簇。聚类之后，可知每一类里有哪些数据，这些数据的最优孕周数的平均值即为所聚的这一类的最优 NIPT 时间点。

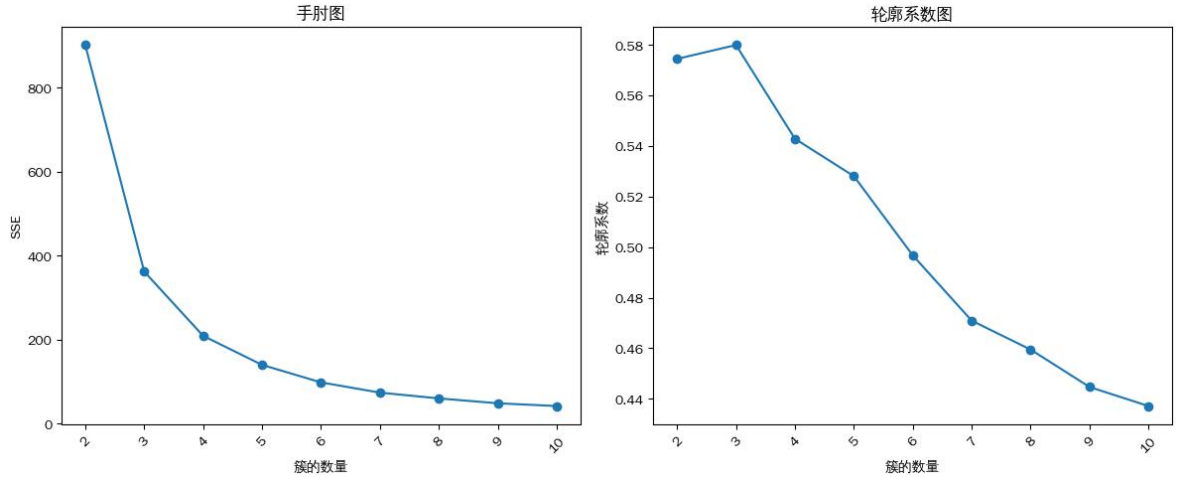


图 9 分类参考图 K-means 聚类

（19-26.29-38.36-40-47）

最终得到分类方案和最优检测孕周数：极低 BMI 组（19-26.2），低 BMI 组（26.29-38.36），高 BMI 组（38.36-40），极高 BMI 组（40-47），分别对应最佳检测时间 10.73 周，15.40 周，22.67 周，30 周，理论极高 BMI 人群最优检测孕周数靠后，考虑到可以多次测量，建议这类人群适当提前检测并多次检测。

6.3.3 检测误差对结果的影响

1. 检验误差可能来源于：

①多种因素(身高、体重、生日的阴历阳历等)的测量误差；

②Y 染色体浓度的检验误差。

2. 对结果的影响：

①如果检验 Y 染色体浓度有误差，可能会导致检测值与真实值的偏离，这时候我们可能会误判它为假阴性或假阳性。（比如真实值是 0.04，但由于检测误差导致观测到的值是 0.038，则误判为假阴性）。

②如果所测量的身高或体重存在误差，导致 BMI 有误差，可能会导致它的分组错误，致使检测时间滞后或提前。若检测时间滞后，则治疗的窗口期风险增大；若检测时间提前，则浓度还未达到 4%，NIPT 正确率不高。

6.4 问题四模型的建立与求解

6.4.1 模型的建立

- **核心特征筛选**

问题四中，需综合纳入 X 染色体、上述染色体的 Z 值、GC 含量、读段数与相关比例以及 BMI 等因素进行分析。基于数据关联性分析，筛选 8 类核心特征构建模型输入维度，确保特征与女胎染色体异常的识别目标直接相关。

1. 直接异常指标

13 号、18 号、21 号染色体的 Z 值

作用：染色体非整倍体一般被界定为：染色体拷贝数偏离正常的两个，呈现为一个或三个；此外，每条染色体的采集读段数量与其长度之间存在正比例关联。13 号、18 号、21 号染色体的 Z 值可以直接反映染色体剂量异常，是异常判断的核心依据。

2.性别适配指标

X 染色体 Z 值

作用：女胎 X 染色体无异常的参考维度。

3.检测质量指标

① X 染色体浓度

作用：评估检测数据的准确性，浓度异常可能会导致检测结果失真。

② 13 号、 18 号、21 号染色体 GC 含量

作用：作为测序质量控制的关键指标。

4.个体差异指标

BMI

作用：用于研究个体差异因素。

- **异常值处理**

1. 染色体数量异常标记

当 13 号、18 号、21 号染色体显示为“非整倍体”时，说明这些染色体存在数量异常；当显示为空白时，则意味着这些染色体的数量正常。在数据预处理时，已提取 AB 列内容，并将其拆分为“染色体异常 13”“染色体异常 17”“染色体异常 21”三列并进行编码，“无异常”“有异常”分别编码为 0 和 1。

因此，只要 13 号、18 号、21 号染色体中任一异常，此行样本数据即打上异常标签，被判定为异常。

2. 测序质量异常标记

GC 含量是测序质量的重要指标。13 号、18 号、21 号染色体任一 GC 值异常，则判定为测序质量异常。

因为所有女胎样本的 13 号、18 号、21 号染色体 GC 含量大多在 0.4 以下，使用 0.4-0.6 的原通用阈值判定会导致大量误判的出现，不合适。因此，使用“均值±2 倍标准差”的统计方法定义其正常范围，超出范围的样本标记为“测序质量异常”（二元特征：1=异常，0=正常）。这样既符合数据分布特性，又能够保证质量判断的客观性与可复现性。

3. X 染色体浓度异常标记

X 染色体浓度显著为负（＜-0.05）时，可能暗示异常，打上异常标签。

6.4.2 模型的求解

- **特征标准化**

Z 值本身是标准化指标，但可能因为计算方式而存在微小差异，因此需要通过 StandardScaler 将 13 号、18 号、21 号染色体的 Z 值进一步标准化。从而消除不同 Z 值特征的量纲差异，使模型更容易收敛。

Z值（$Z-score$）的计算公式：

$$Z = \frac{X - \mu}{\sigma} \tag{4.1}$$

Z再次标准化的计算公式：

$$\frac{Z - \mu}{\sigma} \tag{4.2}$$

- **交互特征构建，通过特征组合捕捉原始特征未体现的潜在模式**

1. 使用 Z 值总和

综合 13 号、18 号、21 号染色体的异常程度，进行加总，反映整体异常风险。

2. GC 偏差值

衡量样本 GC 的含量与总体均值的偏离程度，间接反映测序质量的稳定性。

- **数据平衡与划分**

SMOTE 过采样：调用 SMOTE 对筛选后的特征数据（X）与标签（y）进行过采样，合成少数类样本。通过此操作，两类样本的比例可以更加接近，以此来平衡异常样本数据过少的问题。

数据集划分：采用 7:3 的经典配比，将平衡后的数据集划分为训练集与测试集，并通过编码确保训练集与测试集的正负样本比例与原平衡数据一致，从而避免抽样偏差；同时设置固定随机种子，以保证划分结果的可复现性。

- **基模型选取**

选用逻辑回归、决策树、SVM 三类基模型，分别执行网格搜索训练机器学习。其核心原因在于它们的“多样性”和“互补性”。

- **多样性分析**

多样性方面，三者分属于完全不同的算法家族：逻辑回归是线性模型，决策树属于树模型，SVM 是核方法。它们的学习机制存在本质差异——线性模型主要通过对特征线性加权实现分类，树模型基于特征空间分割，SVM 基于间隔最大化，这三种

模型的学习机制形成“线性－非线性－复杂边界”的全覆盖，能捕捉染色体异常检测中可能存在的各种数据模式，避免了同类模型因机制相似导致的集成效果有限问题。

- **互补性分析**

互补性方面，三者在能力上形成有效补充：逻辑回归擅长捕捉线性关系，决策树擅长捕捉非线性和局部规则，SVM 擅长捕捉复杂边界；同时，它们在处理非线性、适应数据分布、平衡过拟合风险等方面各有侧重，能相互弥补单一模型的局限性，共同提升集成模型的整体性能。

- **基模型训练与参数优化**

1. 扩大参数网格

模型的参数会直接影响性能。扩大参数网格、形成全面的参数网格不仅能够覆盖更多潜在最优解，也让模型更好适配数据特性。因此，为每个基模型设置一个更广泛的参数组合，扩大这三类基模型的参数网格，寻找最优配置。

2. 网格搜索训练基模型

通过编码遍历参数网格，通过 5 折交叉验证（cv=5），找到每个基模型的最优参数组合。

3. 堆叠模型构建，元模型训练与预测

堆叠集成的核心逻辑是用模型预测结果作为新特征，利用不同模型实现优势互补。

- 逻辑回归擅长捕捉线性关系；
- 决策树擅长捕捉非线性和局部规则；
- SVM 擅长捕捉复杂边界。

基模型的预测结果比原始特征更接近目标。将三个基模型的预测结果（0/1）作为新特征，供元模型学习。通过元模型学习基模型预测结果与真实标签的关系，可以得到最终分类。

- **模型性能评估**

在二分类任务中，存在阳性（正）样例 P 和阴性（负）样例 N 两类样本，四种预测结果定义如下：

- **True Positive（TP，真阳性）**：实际为正样本（P），预测结果也为正样本，代表被正确识别的正样本数量。
- **False Negative（FN，假阴性）**：实际为正样本（P），预测结果却为负样本，代表被误判为负样本的正样本数量，即漏报数量。
- **False Positive（FP，假阳性）**：实际为负样本（N），预测结果却为正样本，代表被误判为正样本的负样本数量，即误报数量。
- **True Negative（TN，真阴性）**：实际为负样本（N），预测结果也为负样本，代表被正确识别的负样本数量。

所以有：

$$P = TP + FN$$
$$N = FP + TN \quad (4.3)$$

机器学习得出结果如下：

- 整体分类正确的比例，即准确率达到 88.62%

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.4)$$

- 实际异常样本中被正确识别的比例，即召回率达到 89.36%

$$recall = TPR = \frac{TP}{TP + FN} = \frac{TP}{P} \quad (4.5)$$

- 预测为异常的样本中实际异常的比例，即精确率达到 87.50%，误诊率较低

$$precision = \frac{TP}{TP + FP} \quad (4.6)$$

七、 模型的评价、改进与推广

7.1 模型的优点

1. 模型具有开创性，不适用普通的回归或拟合。在弱相关数据情形下，该模型具有优势。
2. 模型对于稠密数据处理效果较好。
3. 对于离散程度较大的数据，本模型具有处理优势。
4. 模型逻辑连贯，层次递进。从问题一的基础分布建模到问题二、问题三的临床优化、个体差异拓展，再到问题四的多模型集成，形成 “理论－应用－优化－集成”的完整链条，符合科学分析逻辑。

7.2 模型的不足与改进

1. 模型无法做到直接预测 Y 染色体浓度，仅可做到预测其分布。

2. 该模型复杂程度较高，导致单次求解耗费算力更大，耗时相对较长。在数据量较大的情景下，现有方案易出现计算延迟，可能需要通过删减冗余特征、引入分布式计算框架等方式进一步简化模型或改进核心算法，从而有效提升计算效率，满足实时性需求。

7.3 模型的推广

该模型不仅能为围产医学领域提供有效辅助，更具备多维度的推广潜力。在医疗领域，可适配肿瘤标志物检测、心血管疾病预测等多结构化数据场景，复用“多模型+过采样”框架解决样本不平衡问题；对罕见病等中小型数据集场景，其 SVM、决策树等基模型比需要海量数据的深度学习更加稳定。在跨领域应用中，还可拓展至金融风控、工业质检等需分层解释的领域。通过逻辑回归、决策树等提供可解释性，借助 SVM 提升识别能力，从而满足双重需求；同时，其集成学习通用范式仅需要调整参数，即可适配客户流失预测等不同分类任务的数据特性。

八、 参考文献

[1] 国家卫生健康委员会.(2022).产前诊断技术管理办法（2022 年修订）[Z].中华人民共和国国家卫生健康委员会官网.

[2] 中国北方新生儿协作网多中心协作组.极早产儿的早产原因及其对住院期间不良结局的影响：多中心前瞻性队列研究[J].中华围产医学杂志,2023,26(5):357-365.DOI:10.3760/cma.j.cn113903-20221223-01047.

[3] 杨呈勇,甘海燕.一种检测胎儿染色体非整倍体的方法及系统[P].广东：CN105483229B,2018-10-16.

[4] 豆包，豆包大模型 1.6-flash，字节跳动，2025-9-5，9-6，9-7.

[5] Deepseek，Deepseek-V3，深度求索，2025-9-5，9-6，9-7.

附录

附录 1

介绍：该代码是 MATLAB 语言编写，用于进行数据预处理（清洗数据，保留选定待用指标）

```
%男胎数据预处理
only_num= data{:,'VarName15'};
x=1:1082;
y=[];
for ii=x
if only_num(ii) <3000000
y=[y,ii];
end
end
only_num(y)=[];
data_1=data;
data_1(y,:)=[];
GC=data_1{:,'GC'};
x=1:851;
y=[];
z=0;
for ii=x
if GC(ii) <0.395
y=[y,ii];
elseif GC(ii) >0.6
y=[y,ii];
else
z=1;
end
end
GC(y)=[];
data_1(y,:)=[];
only_num(y)=[];
%年龄
age= data_1{:,'VarName3'};
%孕周
week_=data_1{:,'VarName10'};
week=string(week_);
week_1=extractBefore(week,'w');
week_1 = str2double(week_1);
week_2=ones(822,1);
week_2=string(week_2);
x=1:822
```

```
for ii=x
if strcmp(extractAfter(week(ii),"w"),"")
week_2(ii)="0";
else
week_2(ii)=extractAfter(week(ii),4);
end
end
week_2 = str2double(week_2);
week=week_1+week_2/7;
%怀孕次数
pg_num=data_1{:,'VarName29'};
replacementValue = 3;
nanIndices = find(isnan(pg_num));
pg_num(nanIndices) = replacementValue;
%生产次数
pr_num=data_1{:,'VarName30'};
%身高体重
height=data_1{:,'VarName4'};
weight=data_1{:,'VarName5'};
%妊娠方式
IVF=data_1{:,'IVF'};
IVF=string(IVF);
x=1:822;
for ii=x
if strcmp(IVF(ii),"自然受孕")
IVF(ii)="1";
elseif strcmp(IVF(ii),"IUI（人工授精）")
IVF(ii)="2";
else
IVF(ii)="3";
end
end
IVF=str2double(IVF);
%Y 染色体浓度
y_con= data_1{:,'Y'};
bmi=data_1{:,'BMI'};
%序号代码
code=data_1{:,'VarName2'};
code=string(code);
code=extractAfter(code,'A');
code=str2double(code);
spem=[code,y_con,age,pg_num,pr_num,height,weight,bmi,week];
% 指定用于检测离群值的列
column_index = 2;
```

```
    % 设置 IQR 乘数
    k = 1.5;
    % 提取指定列的数据
    target_column = spem(:, column_index);
    % 计算四分位数和 IQR
    Q1 = prctile(target_column, 25);
    Q3 = prctile(target_column, 75);
    IQR_val = Q3 - Q1;
    % 计算正常值的上下界
    lower_bound = Q1 - k * IQR_val;
    upper_bound = Q3 + k * IQR_val;
    % 找出指定列中的离群值
    is_outlier_in_column = target_column < lower_bound | target_column >
upper_bound;
    % 获取非离群值的行索引
    normal_rows = ~is_outlier_in_column;
    % 保留非离群值对应的所有行
    spem = spem(normal_rows, :);
    % 创建行和列的名称
    rowNames = arrayfun(@(x) sprintf('样本%d', x), 1:size(spem, 1),
'UniformOutput', false)';
    colNames = {'代码','Y 染色体浓度', '年龄', '怀孕次数', '生产次数', '身高','体
重','bmi','孕周数'};
    % 将矩阵转换为表格
    dataTable = array2table(spem, 'VariableNames', colNames, 'RowNames',
rowNames);
    % 保存为 Excel 文件
    writetable(dataTable, '预处理后.xlsx', 'WriteRowNames', true);


    %女胎数据预处理
    only_num2= data2{:,'VarName15'};
    x=1:605;
    y11=[];
    for ii=x
    if only_num2(ii) <3000000
    y11=[y11,ii];
    end
    end
    only_num2(y11)=[];
    data2_1=data2;
    data2_1(y11,:)=[];
    GC2=data2_1{:,'GC'};
    x=1:478;
```

```
y111=[];
z=0;
for ii=x
if GC2(ii) <0.395
y111=[y111,ii];
elseif GC2(ii) >0.6
y111=[y111,ii];
else
z=1;
end
end
GC2(y111)=[];
data2_1(y111,:)=[];
only_num2(y111)=[];
%年龄
age2= data2_1{:,'VarName3'};
%孕周
week2_=data2_1{:,'VarName10'};
week2=string(week2_);
week2_1=extractBefore(week2,'w');
week2_1 = str2double(week2_1);
week2_2=ones(462,1);
week2_2=string(week2_2);
x=1:462
for ii=x
if strcmp(extractAfter(week2(ii),"w"),"")
week2_2(ii)="0";
else
week2_2(ii)=extractAfter(week2(ii),4);
end
end
week2_2 = str2double(week2_2);
week2=week2_1+week2_2/7;
%怀孕次数
pg_num2=data2_1{:,'VarName29'};
replacementValue = 3;
nanIndices = find(isnan(pg_num2));
pg_num2(nanIndices) = replacementValue;
%生产次数
pr_num2=data2_1{:,'VarName30'};
%身高体重
height2=data2_1{:,'VarName4'};
weight2=data2_1{:,'VarName5'};
%妊娠方式
```

```
IVF2=data2_1{:,'IVF'};
IVF2=string(IVF2);
x=1:462;
for ii=x
if strcmp(IVF2(ii),"自然受孕")
IVF2(ii)="1";
elseif strcmp(IVF2(ii),"IUI（人工授精）")
IVF2(ii)="2";
else
IVF2(ii)="3";
end
end
IVF2=str2double(IVF2);
%BMI
bmi2=data2_1{:,'BMI'};
%序号代码
code2=data2_1{:,'VarName2'};
code2=string(code2);
code2=extractAfter(code2,'B');
code2=str2double(code2);

% 分类数据存储在 categoricalData 中
abnor= data2{:,'VarName28'};
% 将分类数据转换为字符串数组
strData = string(abnor);
% 初始化结果矩阵/数组
numResults = zeros(length(strData), 2);
strResults = strings(length(strData), 2);
% 遍历每个字符串
for i = 1:length(strData)
currentStr = strData(i);
% 跳过缺失值
if ismissing(currentStr) || strlength(currentStr) == 0
continue;
end
% 使用正则表达式提取所有数字
numbers = regexp(currentStr, '\d+', 'match');
% 根据提取到的数字数量进行处理
switch length(numbers)
case 1
% 只有一个数字，放在第一列
numResults(i, 1) = str2double(numbers{1});
strResults(i, 1) = numbers{1};
case 2
```

```
    % 有两个数字，分别放在两列
    numResults(i, 1) = str2double(numbers{1});
    numResults(i, 2) = str2double(numbers{2});
    strResults(i, 1) = numbers{1};
    strResults(i, 2) = numbers{2};
    otherwise
    % 有多个数字，只取前两个
    if length(numbers) > 2
    numResults(i, 1) = str2double(numbers{1});
    numResults(i, 2) = str2double(numbers{2});
    strResults(i, 1) = numbers{1};
    strResults(i, 2) = numbers{2};
    end
    end
    end
    % 显示结果
    % disp('数值矩阵结果 (缺失位置用 0 填充):');
    % disp(numResults);
    % disp('字符串数组结果 (缺失位置用空字符串填充):');
    % disp(strResults);
    abnor=numResults;
    x_=1:605;
    y_=[];
    for ii=x_
    if abnor(ii,1)+abnor(ii,2)==0
    y=[y,ii];
    end
    end
    columnNames = {'VarName12', 'VarName13',
'VarName14','VarName15','GC','Z','Z1','Z2','XZ','X','GC1','GC2','GC3','Va
rName27'};
    total_ = data2{:, columnNames};
    % 获取数据行数
    N = size(abnor, 1);
    % 初始化结果矩阵
    resultMatrix = zeros(N, 3); % N×3 矩阵，初始化为全 0
    % 遍历每一行数据
    for i = 1:N
    % 获取当前行的所有非零数字
    currentNumbers = abnor(i, :);
    nonZeroNumbers = currentNumbers(currentNumbers ~= 0);
    % 检查数字 13 是否存在
    if any(nonZeroNumbers == 13)
    resultMatrix(i, 1) = 1;
```

```
    end
    % 检查数字 18 是否存在
    if any(nonZeroNumbers == 18)
    resultMatrix(i, 2) = 1;
    end
    % 检查数字 21 是否存在
    if any(nonZeroNumbers == 21)
    resultMatrix(i, 3) = 1;
    end
    end
    abnor=resultMatrix;
    total_=[total_,abnor];
    total_(y11,:)=[];
    total_(y111,:)=[];
    excel_=[code2,age2,height2,weight2,IVF2,week2,bmi2,total_,pg_num2,pr_num2];
    % 创建行和列的名称
    rowNames = arrayfun(@(x) sprintf('样本%d', x), 1:size(excel_, 1), 'UniformOutput', false)';
    colNames = {'代码', '年龄','身高','体重','妊娠方式', '孕周数','bmi','原始读段数','在参考基因组上比对的比例','重复读段的比例','唯一比对的读段数','GC 含量','13 号染色体 z 值','18 号染色体 z 值','21 号染色体 z 值','X 染色体 z 值','X 染色体浓度','13 号 GC','18 号 GC','21 号 GC','被过滤掉比例','染色体异常 13','染色体异常 18','染色体异常 21','怀孕次数', '生产次数'};
    % 将矩阵转换为表格
    dataTable = array2table(excel_, 'VariableNames', colNames, 'RowNames', rowNames);
    % 保存为 Excel 文件
    writetable(dataTable, '女预处理后（全）(4).xlsx', 'WriteRowNames', true);
```

附录 2

介绍：该代码是 python 编写的，做出 Y 浓度，孕周数，bmi,体重，身高，年龄的斯皮尔曼系数热力图

```
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import spearmanr
# 提取需要计算相关系数的列
columns = ['Y 染色体浓度', '年龄', '身高', '体重', 'bmi', '孕周数']
data = df[columns]
# 计算斯皮尔曼相关系数和 p 值
corr_matrix, p_matrix = spearmanr(data)
# 将结果转换为 DataFrame 方便查看
corr_df = pd.DataFrame(corr_matrix, index=columns, columns=columns)
p_df = pd.DataFrame(p_matrix, index=columns, columns=columns)
print("斯皮尔曼相关系数矩阵：")
print(corr_df.round(2))
print("\n 斯皮尔曼相关系数的 p 值矩阵：")
print(p_df.round(2))
# 设置图片清晰度
plt.rcParams['figure.dpi'] = 300
# 设置中文字体
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
# 绘制热力图
plt.figure(figsize=(10, 8))
sns.heatmap(corr_df, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('斯皮尔曼相关系数热力图')
plt.show()
```

附录 3

介绍：该代码是 python 编写的，作用是计算 GAMMA 分布和 Beta 分布的 AIC

```
from scipy.stats import gamma as gamma_distribution
from scipy.stats import beta as beta_distribution

# 计算 GAMMA 分布的 AIC
# 使用极大似然估计法估计 Gamma 分布的参数
gamma_alpha, gamma_loc, gamma_beta = gamma_distribution.fit(df['Y 染色体浓度'])
# 计算似然函数值
log_likelihood_gamma = np.sum(gamma_distribution.logpdf(df['Y 染色体浓度'],
gamma_alpha, gamma_loc, gamma_beta))
# 参数数量
k_gamma = 3
# 计算 AIC
aic_gamma = 2 * k_gamma - 2 * log_likelihood_gamma
```

```
# 计算 Beta 分布的 AIC
# 由于 Beta 分布的取值范围在 [0, 1] 之间，需要对 Y 染色体浓度列进行归一化
y_chromosome_concentration = df['Y 染色体浓度']
y_chromosome_concentration_normalized = (y_chromosome_concentration - y_chromosome_concentration.min()) / (
        y_chromosome_concentration.max() - y_chromosome_concentration.min())

# 使用极大似然估计法估计 Beta 分布的参数
beta_a, beta_b, beta_loc, beta_scale = beta_distribution.fit(y_chromosome_concentration_normalized)

# 计算似然函数值
log_likelihood_beta = np.sum(beta_distribution.logpdf(y_chromosome_concentration_normalized, beta_a, beta_b, beta_loc, beta_scale))

# 参数数量
k_beta = 4

# 计算 AIC
aic_beta = 2 * k_beta - 2 * log_likelihood_beta

print('GAMMA 分布的 AIC 值为:', aic_gamma)
print('Beta 分布的 AIC 值为:', aic_beta)
```

附录 4[4]

介绍：该代码是 python 编写的，作用是双参数极大似然估计参数

```
import pandas as pd
import numpy as np
from scipy.optimize import minimize
from scipy.special import gamma, digamma, polygamma
from scipy.stats import norm

# 1. 读取并准备数据
excel_file = pd.ExcelFile(r"C:\Users\29813\Desktop\预处理.xlsx")
df = excel_file.parse('Sheet1')

# 提取自变量（X）和因变量（y）：因变量需为正数（Gamma 分布要求）
X = df[['bmi', '孕周数']].values  # 2 个自变量
```

```
y = df['Y 染色体浓度'].values  # 因变量（Y 染色体浓度）
n_samples = len(y)
n_features = X.shape[1]  # 2

# 2. 构建负对数似然函数（优化目标：最小化负对数似然）
def neg_log_likelihood(params, X, y):
    """
    params: 待估计的参数向量，共 6 个参数：
        [β0, β1, β2,  # α的回归系数（3 个）
         γ0, γ1, γ2]  # λ的回归系数（3 个）
    X: 自变量矩阵 (n_samples, 2)
    y: 因变量向量 (n_samples,)
    """
    # 拆分参数：α的系数（β）和λ的系数（γ）
    beta = params[:3]  # α的截距+2 个自变量系数
    gamma_coef = params[3:]  # λ的截距+2 个自变量系数

    # 计算每个样本的 α 和 λ（通过对数链接确保为正）
    alpha = np.exp(beta[0] + X @ beta[1:])  # X@beta[1:] = β1×bmi + β2×孕周数
    lam = np.exp(gamma_coef[0] + X @ gamma_coef[1:])  # λ = αθ

    # 计算 Gamma 分布的对数似然（单个样本）
    # PDF 的对数形式：(α-1)*ln(x) - (αx)/λ - α*ln(λ/α) - ln(Γ(α))
     log_likelihood = (alpha - 1) * np.log(y) - (alpha * y) / lam - alpha * np.log(lam / alpha) - np.log(gamma(alpha))

    # 返回负对数似然的总和（优化器需要最小化）
    return -np.sum(log_likelihood)

# 3. 手动实现近似导数计算 Hessian 矩阵
def compute_std_errors(params, X, y):
    epsilon = 1e-8
    n_params = len(params)
    hess = np.zeros((n_params, n_params))

    for i in range(n_params):
        for j in range(n_params):
            x_plus_epsilon_i = params.copy()
            x_plus_epsilon_i[i] += epsilon
            if i == j:
                x_plus_epsilon_j = x_plus_epsilon_i.copy()
                x_plus_epsilon_j[j] += epsilon
                x_minus_epsilon_j = params.copy()
          x_minus_epsilon_j[j] -= epsilon
```

```
                else:
                    x_plus_epsilon_j = x_plus_epsilon_i.copy()
                    x_plus_epsilon_j[j] += epsilon
                    x_minus_epsilon_j = x_plus_epsilon_i.copy()
                    x_minus_epsilon_j[j] -= epsilon

                f_plus_plus = neg_log_likelihood(x_plus_epsilon_j, X, y)
                f_plus_minus = neg_log_likelihood(x_minus_epsilon_j, X, y)

                hess[i, j] = (f_plus_plus - f_plus_minus) / (2 * epsilon)

    try:
        cov_matrix = np.linalg.inv(hess)
        std_errors = np.sqrt(np.diag(cov_matrix))
    except np.linalg.LinAlgError:
        std_errors = np.full_like(params, np.nan)

    return std_errors

# 1. 设置初始参数值（基于单参数模型的系数，确保优化收敛）
# 初始值：α的系数先设为常数（β1-β2=0，β0=ln(8.01)）；λ的系数用之前单参数模型的结果
initial_beta = [np.log(8.01), 0, 0]  # α的初始系数（先假设α不随自变量变化）
initial_gamma = [-6.9170, 0.0785, 0.0107]  # λ的初始系数（单参数模型结果）
initial_params = initial_beta + initial_gamma  # 合并为 6 个初始参数

# 2. 执行极大似然优化（L - BFGS - B 算法：支持边界约束，确保数值稳定）
result = minimize(
    fun=neg_log_likelihood,
    x0=initial_params,
    args=(X, y),
    method='L-BFGS-B',
    options={'maxiter': 1000, 'disp': True}  # 最大迭代次数 1000，显示优化过程
)

# 3. 提取优化结果
if result.success:
    estimated_params = result.x  # 估计的 6 个参数
    min_neg_log_like = result.fun  # 最小化的负对数似然值
    # 计算标准误差
    std_errors = compute_std_errors(estimated_params, X, y)
else:
    print("优化未收敛！错误信息：", result.message)
    estimated_params = None
```

```
    std_errors = None

# 整理结果为 DataFrame，便于解读
if result.success:
    # 定义参数名称
    param_names = [
        'α_截距(β0)', 'α_bmi(β1)', 'α_孕周数(β2)',
        'λ_截距(γ0)', 'λ_bmi(γ1)', 'λ_孕周数(γ2)'
    ]

    # 计算 z 值（系数/标准误差）和 p 值（基于正态分布近似）
    z_values = estimated_params / std_errors
    p_values = 2 * (1 - norm.cdf(np.abs(z_values)))  # 双侧检验 p 值

    # 构建结果表
    result_df = pd.DataFrame({
        '参数名称': param_names,
        '估计值': np.round(estimated_params, 4),
        '标准误差': np.round(std_errors, 4),
        'z 值': np.round(z_values, 3),
        'p 值': np.round(p_values, 4)
    })

    # 输出模型拟合指标
    print("=" * 50)
    print("双参数 Gamma 回归模型拟合结果")
    print("=" * 50)
    print(f"最小负对数似然值: {np.round(min_neg_log_like, 2)}")
      print(f"AIC (Akaike 信息准则): {np.round(2 * len(estimated_params) + 2 *
min_neg_log_like, 2)}")
        print(f"BIC (Bayesian 信息准则): {np.round(len(estimated_params) *
np.log(n_samples) + 2 * min_neg_log_like, 2)}")
    print("\n 参数估计结果:")
    print(result_df.to_string(index=False))
```

附录 5[5]

介绍：该代码是 MATLAB 语言编写，用于求解第二问目标函数最大值

```
    %第二问
    %先求分布函数最值，对（1-CDF）进行归一化
    % 定义 BMI 和 t 的范围
    bmi_min = 20;
    bmi_max = 47;
    t_min =10;
    t_max =30;
    % 定义网格点数量
    num_points = 200;
    % 创建网格
    bmi_vec = linspace(bmi_min, bmi_max, num_points);
    t_vec = linspace(t_min, t_max, num_points);
    [BMI, T] = meshgrid(bmi_vec, t_vec);
    % 计算 k 和 z
    k = exp(2.0619 - 0.2056 * BMI + 0.1183 * T);
    z = 0.04 * exp(8.9664 - 0.6797 * BMI - 0.1148 * T);
    % 计算 Gamma CDF 值 P = gammainc(z, k)
    P = gammainc(z, k);
    % 找到最小值和最大值
    [min_val, min_idx] = min(P(:));
    [max_val, max_idx] = max(P(:));
    % 找到对应的 BMI 和 t 值
    [min_row, min_col] = ind2sub(size(P), min_idx);
    min_bmi = BMI(min_row, min_col);
    min_t = T(min_row, min_col);
    [max_row, max_col] = ind2sub(size(P), max_idx);
    max_bmi = BMI(max_row, max_col);
    max_t = T(max_row, max_col);
    % 输出结果
    fprintf('在 BMI 范围 [%.1f, %.1f] 和 t 范围 [%.1f, %.1f] 内:\n', bmi_min,
bmi_max, t_min, t_max);
    fprintf('表达式最小值: %.6f\n', min_val);
    fprintf('最小值出现在 BMI = %.4f, t = %.4f\n', min_bmi, min_t);
    fprintf('表达式最大值: %.6f\n', max_val);
    fprintf('最大值出现在 BMI = %.4f, t = %.4f\n', max_bmi, max_t);
    % 可视化
    figure;
    surf(BMI, T, P);
    xlabel('BMI');
    ylabel('t');
    zlabel('CDF 值');
    title('CDF 值随 BMI 和 t 的变化');
    colorbar;
    hold on;
```

```
    plot3(min_bmi, min_t, min_val, 'ro', 'MarkerSize', 10,
'MarkerFaceColor', 'r');
    plot3(max_bmi, max_t, max_val, 'go', 'MarkerSize', 10,
'MarkerFaceColor', 'g');
    legend('CDF 曲面', '最小值点', '最大值点');
    %计算目标函数最大值，寻找最佳 NIPT 时间
    % 定义参数范围
    bmi_min = 20;
    bmi_max = 47;
    t_min = 10;
    t_max = 30;
    % 使用优化算法寻找最优 t
    % 对于每个可能的 BMI 值，找到最优 t
    bmi_values = linspace(bmi_min, bmi_max, 50);
    optimal_t_values = zeros(size(bmi_values));
    max_values = zeros(size(bmi_values));
    for i = 1:length(bmi_values)
    bmi = bmi_values(i);
    % 定义针对当前 bmi 的优化函数
    opt_func = @(t) -composite_func(bmi, t);
    % 在 t 范围内寻找最小值
    [t_opt, f_val] = fminbnd(opt_func, t_min, t_max);
    optimal_t_values(i) = t_opt;
    max_values(i) = -f_val;
    end
    % 找到最优解
    [global_max_val, global_max_idx] = max(max_values);
    global_optimal_t = optimal_t_values(global_max_idx);
    global_optimal_bmi = bmi_values(global_max_idx);
    % 输出结果
    fprintf('在 BMI 区间 [%.1f, %.1f] 和 t 范围 [%.1f, %.1f] 内:\n', bmi_min,
bmi_max, t_min, t_max);
    fprintf('复合表达式最大值 %.6f 出现在 t = %.4f 和 BMI = %.4f 处\n', ...
    global_max_val, global_optimal_t, global_optimal_bmi);
    % 可视化结果
    figure;
    % 最优 t 随 BMI 的变化
    subplot(2, 1, 1);
    plot(bmi_values, optimal_t_values, 'b-', 'LineWidth', 2);
    xlabel('BMI');
    ylabel('最优 t 值');
    title('最优 t 值随 BMI 的变化');
    grid on;
    %  最大表达式值随 BMI 的变化
```

```
    subplot(2, 1, 2);
    plot(bmi_values, max_values, 'r-', 'LineWidth', 2);
    hold on;
    plot(global_optimal_bmi, global_max_val, 'ro', 'MarkerSize', 10,
'MarkerFaceColor', 'r');
    xlabel('BMI');
    ylabel('最大表达式值');
    title('最大表达式值随 BMI 的变化');
    grid on;
    legend('最大值曲线', '全局最大值点');

    % 对于全局最优 BMI，展示表达式随 t 的变化
    figure;
    bmi_global = global_optimal_bmi;
    t_values = linspace(t_min, t_max, 1000);
    g_values = arrayfun(@(t) composite_func(bmi_global, t), t_values);
    plot(t_values, g_values, 'g-', 'LineWidth', 2);
    hold on;
    plot(global_optimal_t, global_max_val, 'ro', 'MarkerSize', 10,
'MarkerFaceColor', 'r');
    xlabel('t');
    ylabel('表达式值');
    title(sprintf('表达式值随 t 的变化 (BMI = %.2f)', bmi_global));
    grid on;
    legend('表达式值', '最大值点');
    % 输出详细结果表
    fprintf('\n 详细结果表:\n');
    fprintf('BMI\t\t 最优 t\t\t 最大值\n');
    fprintf('----\t\t-----\t\t------\n');
    for i = 1:length(bmi_values)
    fprintf('%.2f\t\t%.2f\t\t%.6f\n', bmi_values(i),
optimal_t_values(i), max_values(i));
    end
```

附录 6[4]

介绍：该代码是 python 编写的，作用是检验年龄对 Y 浓度的显著性

```
import pandas as pd
import numpy as np
from scipy.optimize import minimize
from scipy.special import gamma, gammainc
from scipy.stats import norm

# 1. 数据准备
excel_file = pd.ExcelFile('/mnt/预处理.xlsx')
df = excel_file.parse('Sheet1')

# 选择自变量（保留孕周数、体重、年龄，简化模型）
X = df[['孕周数', '体重', '年龄']].values
y = df['Y 染色体浓度'].values
y = np.clip(y, 1e-6, None)  # 避免 y=0 导致 log 错误

# 自变量标准化
X_mean = np.mean(X, axis=0)
X_std = np.std(X, axis=0)
X_norm = (X - X_mean) / X_std
n_samples, n_features = X_norm.shape

# 2. 分步估计函数
def estimate_lambda_params(alpha, X, y):
    """固定 alpha 时，估计 lambda 的参数"""
    def neg_log_lik(gamma_coef):
        lam = np.exp(gamma_coef[0] + X @ gamma_coef[1:])
                    return -np.sum((alpha - 1)*np.log(y) - (alpha*y)/lam -
alpha*np.log(lam/alpha))

    # 初始值
    gamma_init = np.array([np.log(np.mean(alpha*y))] + [0.]*n_features)
    result = minimize(neg_log_lik, gamma_init, method='Nelder-Mead')
    return result.x

def estimate_alpha_params(lam, X, y):
    """固定 lambda 时，估计 alpha 的参数"""
    def neg_log_lik(beta):
        alpha = np.exp(beta[0] + X @ beta[1:])
        alpha = np.clip(alpha, 1e-3, 100)  # 限制 alpha 范围
        return -np.sum((alpha - 1)*np.log(y) - (alpha*y)/lam - np.log(gamma(alpha)))

    # 初始值
```

```
    beta_init = np.array([np.log(np.mean(y**2 / np.var(y)))] + [0.]*n_features)
    result = minimize(neg_log_lik, beta_init, method='Nelder-Mead')
    return result.x

# 3. 迭代逼近
# 初始值设置
alpha_init = np.full(n_samples, np.log(np.mean(y**2 / np.var(y))))  # 基于二阶矩的初始 alpha
gamma_coef = estimate_lambda_params(alpha_init, X_norm, y)
lam = np.exp(gamma_coef[0] + X_norm @ gamma_coef[1:])

# 迭代 5 次
for i in range(5):
    beta = estimate_alpha_params(lam, X_norm, y)
    alpha = np.exp(beta[0] + X_norm @ beta[1:])
    gamma_coef = estimate_lambda_params(alpha, X_norm, y)
    lam = np.exp(gamma_coef[0] + X_norm @ gamma_coef[1:])
      print(f"迭代{i+1}完成，alpha 均值: {np.mean(alpha):.4f}, lambda 均值: {np.mean(lam):.4f}")

# 4. 计算近似标准误差（基于参数敏感性）
def param_sensitivity(param, idx, X, y, func, base_value):
    """计算参数微小扰动对似然的影响"""
    epsilon = 1e-4
    param[idx] += epsilon
    if func == 'alpha':
        alpha = np.exp(param[0] + X @ param[1:])
        new_lam = np.exp(gamma_coef[0] + X @ gamma_coef[1:])
                ll = np.sum((alpha - 1)*np.log(y) - (alpha*y)/new_lam - np.log(gamma(alpha)))
    else:
        new_lam = np.exp(param[0] + X @ param[1:])
                ll = np.sum((alpha - 1)*np.log(y) - (alpha*y)/new_lam - alpha*np.log(new_lam/alpha))
    return np.abs(ll - base_value)

# 基准似然值
base_alpha = np.exp(beta[0] + X_norm @ beta[1:])
base_lam = np.exp(gamma_coef[0] + X_norm @ gamma_coef[1:])
base_ll = np.sum((base_alpha - 1)*np.log(y) - (base_alpha*y)/base_lam - np.log(gamma(base_alpha)))

# 计算敏感性（近似标准误差）
alpha_sens = [param_sensitivity(beta.copy(), i, X_norm, y, 'alpha', base_ll) for i in
```

```
range(len(beta))]
lambda_sens = [param_sensitivity(gamma_coef.copy(), i, X_norm, y, 'lambda', base_ll) for i in range(len(gamma_coef))]
alpha_se = np.array(alpha_sens) * 0.1  # 缩放因子
lambda_se = np.array(lambda_sens) * 0.1

# 5. 整理结果
param_names = [
    'α_截距(β0)', 'α_孕周数(β1)', 'α_体重(β2)', 'α_年龄(β3)',
    'λ_截距(γ0)', 'λ_孕周数(γ1)', 'λ_体重(γ2)', 'λ_年龄(γ3)'
]
params = np.concatenate([beta, gamma_coef])
std_errors = np.concatenate([alpha_se, lambda_se])

# 近似 z 值和 p 值
z_values = params / std_errors
p_values = 2 * (1 - norm.cdf(np.abs(z_values)))

result_df = pd.DataFrame({
    '参数名称': param_names,
    '近似估计值': np.round(params, 4),
    '近似标准误差': np.round(std_errors, 4),
    'z 值': np.round(z_values, 3),
    'p 值': np.round(p_values, 4)
})

print("\n" + "=" * 50)
print("双参数 Gamma 回归模型近似参数估计结果")
print("=" * 50)
print(result_df.to_string(index=False))

# 6. 输出 CDF 解析式所需参数
print("\n 用于 CDF 解析式的参数：")
print("α参数（β）：", np.round(beta, 4))
print("λ参数（γ）：", np.round(gamma_coef, 4))
```

附录 7
介绍：该代码是 python 编写的，作用是分类
from sklearn.cluster import KMeans

```
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# 提取特征
X = df[['BMI', '最优 t']]

# 使用手肘法确定最佳的簇数
sse = []
silhouette_scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)
    labels = kmeans.labels_
    silhouette_scores.append(silhouette_score(X, labels))

# 绘制手肘图
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(2, 11), sse, marker='o')
plt.xlabel('簇的数量')
plt.xticks(rotation=45)
plt.ylabel('SSE')
plt.title('手肘图')

# 绘制轮廓系数图
plt.subplot(1, 2, 2)
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.xlabel('簇的数量')
plt.xticks(rotation=45)
plt.ylabel('轮廓系数')
plt.title('轮廓系数图')

plt.tight_layout()
plt.show()
# 假设根据图选择最佳簇数为 3
best_k = 3

# 使用最佳簇数进行 KMeans 聚类
kmeans = KMeans(n_clusters=best_k, random_state=42)
df['聚类类别'] = kmeans.fit_predict(X)

# 将结果保存为 Excel 文件
df.to_excel('/mnt/bmi-t-max 表(1)_聚类结果.xlsx', index=False)
```

附录 8[4]

介绍：该代码是 python 编写的，作用是完成第三问的优化

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from scipy.special import gammainc, gamma
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False  # 解决负号显示异常问题
def calculate_alpha(bmi, 孕周数):
    beta0 = 2.0619
    beta1 = -0.2056
    beta2 = 0.1183
    return np.exp(beta0 + beta1 * bmi + beta2 * 孕周数)
def calculate_F(x, bmi, 孕周数):
    # 公式中给定的参数
    beta0 = 2.0619
    beta1 = -0.2056
    beta2 = 0.1183
    gamma0 = -6.9045
    gamma1 = 0.4741
    gamma2 = 0.2331
    gamma_scalar = 0.4741  # 公式中的γ

    # 计算伽马函数的形状参数 a
    a = np.exp(beta0 + beta1 * bmi + beta2 * 孕周数)
    # 计算不完全伽马函数的上界参数
       upper_bound = (x * np.exp(beta0 + beta1 * bmi + beta2 * 孕周数)) /
np.exp(gamma0 + gamma1 * bmi + gamma2 * 孕周数)

    # 不完全伽马函数 γ(a, upper_bound) / Γ(a) = gammainc(a, upper_bound)
    numerator = gamma_scalar * gammainc(a, upper_bound)
    denominator = gamma(a)
    return numerator / denominator
def objective_function(孕周数, bmi):
    x_val = 0.04  # Y 染色体浓度取值
    term1 = 0.2 * (0.04274 - calculate_F(x_val, bmi, 孕周数)) / 0.04274
    term2 = 0.8 * (孕周数 - 10) / 20
    return - (term1 - term2)  # 最大化 term1 - term2 → 最小化其负数
def constraint_function(孕周数, bmi):
    x_val = 0.04
```

```
    return 0.01 - calculate_F(x_val, bmi, 孕周数)  # 要求 F(0.04) ≤ 0.1 → 0.1 - F(0.04) ≥ 0
# 设置 BMI 范围（示例：19~40，覆盖体重 50~110、身高~1.6m 的情况）
bmi_values = np.linspace(19, 40, 50)
optimal_weeks = []

def constraint_alpha(孕周数, bmi):
    alpha = calculate_alpha(bmi, 孕周数)
    return alpha - 0.05  # 约束：α ≥5 → α - 5 ≥0

for bmi in bmi_values:
    initial_week = 20.0
    bounds = [(10, 30)]
    # 约束列表：同时满足两个条件
    constraints = [
        # 原约束：F(0.04) ≤ 0.1 → 0.1 - F(...) ≥ 0
        {'type': 'ineq', 'fun': lambda w, b=bmi: constraint_function(w, b)},
        # 新约束：α ≥5 → α - 5 ≥ 0
        {'type': 'ineq', 'fun': lambda w, b=bmi: constraint_alpha(w, b)}
    ]

    # 执行带约束的优化
    result = minimize(
        objective_function,
        initial_week,
        args=(bmi,),
        bounds=bounds,
        constraints=constraints,
        method='SLSQP'
    )

    if result.success:
        optimal_weeks.append(result.x[0])
    else:
        optimal_weeks.append(np.nan)
        print(f"BMI={bmi:.2f}时优化失败，原因：{result.message}")

optimal_weeks = np.array(optimal_weeks)
plt.figure(figsize=(10, 6))
plt.plot(bmi_values, optimal_weeks, 'b-', linewidth=2)
plt.xlabel('BMI')
plt.ylabel('最优孕周数')
plt.title('不同 BMI 下的最优孕周数')
plt.grid(True)
```

```
plt.show()
```

附录 9[4]

介绍：该代码是 Python 语言编写的，作用是对 BMI 分组

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# 提取特征
X = df[['BMI', '最优孕周数']]

# 使用手肘法确定最佳的簇数
sse = []
silhouette_scores = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)
    labels = kmeans.labels_
    silhouette_scores.append(silhouette_score(X, labels))

# 绘制手肘图
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(2, 11), sse, marker='o')
plt.xlabel('簇的数量')
plt.xticks(rotation=45)
plt.ylabel('SSE')
plt.title('手肘图')

# 绘制轮廓系数图
plt.subplot(1, 2, 2)
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.xlabel('簇的数量')
plt.xticks(rotation=45)
plt.ylabel('轮廓系数')
plt.title('轮廓系数图')

plt.tight_layout()
plt.show()
# 假设根据图选择最佳簇数为 3
```

```
best_k = 3

# 使用最佳簇数进行 KMeans 聚类
kmeans = KMeans(n_clusters=best_k, random_state=42)
df['聚类类别'] = kmeans.fit_predict(X)

# 将结果保存为 Excel 文件
df.to_excel('/mnt/BMI 与最优孕周数_聚类结果.xlsx', index=False)
```

附录 10[5]

介绍：该代码是 Python 语言编写，用于求解以逻辑回归、决策树和 SVM 三模型为基模型的元模型的预测效果

```
#第四问
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, recall_score, precision_score, confusion_matrix
from imblearn.over_sampling import SMOTE

# 读取数据
df = pd.read_excel('/mnt/女预处理后（全）(4).xlsx')

# 异常样本标注（标签：1 = 异常，0 = 正常）
df['标签'] = df[['染色体异常 13', '染色体异常 18', '染色体异常 21']].max(axis=1)

# 特征筛选（核心特征 + 剔除无关特征）
core_features = [
    '13 号染色体 Z 值', '18 号染色体 Z 值', '21 号染色体 Z 值', 'X 染色体 Z 值',
    'X 染色体浓度', '13 号 GC', '18 号 GC', '21 号 GC', 'bmi'
]
df_model = df[core_features + ['标签']].copy()

# 计算 13 号 GC、18 号 GC 和 21 号 GC 的均值和标准差
gc_cols = ['13 号 GC', '18 号 GC', '21 号 GC']
gc_mean = df_model[gc_cols].mean()
gc_std = df_model[gc_cols].std()
```

```
# 根据均值 ±2 倍标准差确定正常范围
lower_bound = gc_mean - 2 * gc_std
upper_bound = gc_mean + 2 * gc_std

# 异常值处理（标记测序质量）
df_model['测序质量异常'] = ((df_model[gc_cols] < lower_bound) | (df_model[gc_cols] > upper_bound)).any(axis=1).astype(int)

# X 染色体浓度异常标记（显著负值：< - 0.05）
df_model['X 浓度异常'] = (df_model['X 染色体浓度'] < -0.05).astype(int)

# 缺失值与异常样本过滤（删除标签缺失样本）
df_model = df_model.dropna(subset=['标签'])

# 特征工程
# Z 值标准化（仅对 13/18/21 号染色体 Z 值）
z_cols = ['13 号染色体 Z 值', '18 号染色体 Z 值', '21 号染色体 Z 值']
scaler_z = StandardScaler()
df_model[z_cols] = scaler_z.fit_transform(df_model[z_cols])

# 交互特征构建
# 整体染色体异常程度（Z 值总和）
df_model['Z 值总和'] = df_model[z_cols].sum(axis=1)
# GC 含量偏差值（与各染色体 GC 含量均值的绝对差均值）
df_model['GC 偏差值'] = df_model[gc_cols].apply(lambda x: np.mean(np.abs(x - gc_mean)), axis=1)

# 最终特征集（加入交互特征，剔除原始 GC 列避免多重共线性）
final_features = [
    '13 号染色体 Z 值', '18 号染色体 Z 值', '21 号染色体 Z 值', 'X 染色体 Z 值',
    'X 染色体浓度', 'bmi', 'Z 值总和', 'GC 偏差值', '测序质量异常', 'X 浓度异常'
]
X = df_model[final_features]
y = df_model['标签']

# 使用 SMOTE 进行过采样
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# 样本划分（7:3）
X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3, random_state=42, stratify=y_resampled
)
```

```
# 扩大逻辑回归的参数网格
lr_param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100, 1000],  # 扩大正则化参数范围
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga'],  # 增加 saga 求解器
    'class_weight': ['balanced', None],  # 增加类别权重选项
    'max_iter': [500, 1000, 2000]  # 增加迭代次数选项
}

# 扩大决策树的参数网格
dt_param_grid = {
    'max_depth': [2, 3, 5, 7, 9, 11, None],  # 增加更深的树和不限制深度选项
    'min_samples_split': [2, 5, 10, 20, 30],  # 扩大分裂样本数范围
    'min_samples_leaf': [1, 2, 4, 8, 16],  # 扩大叶节点样本数范围
    'criterion': ['gini', 'entropy'],  # 增加分裂标准选项
    'splitter': ['best', 'random'],  # 增加分裂策略选项
    'class_weight': ['balanced', None]  # 增加类别权重选项
}

# 扩大 SVM 的参数网格
svm_param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100, 1000],  # 扩大正则化参数范围
    'kernel': ['linear', 'rbf', 'poly', 'sigmoid'],  # 增加 sigmoid 核
    'degree': [2, 3, 4, 5],  # 扩大多项式核阶数范围
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1],  # 增加 gamma 参数选项
    'shrinking': [True, False],  # 增加收缩选项
    'class_weight': ['balanced', None],  # 增加类别权重选项
    'max_iter': [-1, 1000, 2000]  # 增加迭代次数选项
}

# 使用网格搜索寻找逻辑回归的最佳参数
print("正在训练逻辑回归模型...")
lr_grid_search = GridSearchCV(
    LogisticRegression(random_state=42),
    lr_param_grid,
    cv=5,
    n_jobs=-1,  # 使用所有可用 CPU
    verbose=1
)
lr_grid_search.fit(X_train, y_train)
lr_model = lr_grid_search.best_estimator_
print(f"逻辑回归最佳参数: {lr_grid_search.best_params_}")
```

```
# 使用网格搜索寻找决策树的最佳参数
print("\n 正在训练决策树模型...")
dt_grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    dt_param_grid,
    cv=5,
    n_jobs=-1,
    verbose=1
)
dt_grid_search.fit(X_train, y_train)
dt_model = dt_grid_search.best_estimator_
print(f"决策树最佳参数: {dt_grid_search.best_params_}")

# 使用网格搜索寻找 SVM 的最佳参数
print("\n 正在训练 SVM 模型...")
svm_grid_search = GridSearchCV(
    SVC(random_state=42),
    svm_param_grid,
    cv=5,
    n_jobs=-1,
    verbose=1
)
svm_grid_search.fit(X_train, y_train)
svm_model = svm_grid_search.best_estimator_
print(f"SVM 最佳参数: {svm_grid_search.best_params_}")

# 用基模型对训练集和测试集进行预测，作为新特征
X_train_new = np.column_stack((
    lr_model.predict(X_train),
    dt_model.predict(X_train),
    svm_model.predict(X_train)
))

X_test_new = np.column_stack((
    lr_model.predict(X_test),
    dt_model.predict(X_test),
    svm_model.predict(X_test)
))

# 训练元模型（使用逻辑回归）
meta_model = LogisticRegression(random_state=42, max_iter=1000)
meta_model.fit(X_train_new, y_train)

# 元模型进行预测
```

```
y_pred = meta_model.predict(X_test_new)

# 性能评估
accuracy = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)

# 输出关键结果
print("\n===== 模型堆叠（Stacking）性能 =====")
print(f"准确率: {round(accuracy, 4)}")
print(f"召回率: {round(recall, 4)}")
print(f"精确率: {round(precision, 4)}")
print("\n 测试集混淆矩阵：")
print(confusion_matrix(y_test, y_pred))

# 输出各基模型的单独性能
print("\n===== 各基模型单独性能 =====")
lr_pred = lr_model.predict(X_test)
print(f"逻辑回归准确率: {round(accuracy_score(y_test, lr_pred), 4)}")

dt_pred = dt_model.predict(X_test)
print(f"决策树准确率: {round(accuracy_score(y_test, dt_pred), 4)}")

svm_pred = svm_model.predict(X_test)
print(f"SVM 准确率: {round(accuracy_score(y_test, svm_pred), 4)}")
```

附录 11

介绍：问题二模型求解

Row	BMI	最优 t	最大值
样本 1	20	11.16040173	0.084248447
样本 2	20.55102041	11.70669225	0.061809106
样本 3	21.10204082	12.26949933	0.038754054
样本 4	21.65306122	12.84764201	0.015124984
样本 5	22.20408163	13.44012197	-0.009040132
样本 6	22.75510204	14.04594992	-0.033706665
样本 7	23.30612245	14.66425468	-0.058842979
样本 8	23.85714286	15.29426828	-0.084420134
样本 9	24.40816327	15.93525659	-0.110411605
样本 10	24.95918367	16.58654798	-0.136793044
样本 11	25.51020408	17.24754798	-0.163542064
样本 12	26.06122449	17.91768195	-0.190638043
样本 13	26.6122449	18.5964681	-0.218061958
样本 14	27.16326531	19.28339869	-0.245796231
样本 15	27.71428571	19.9780368	-0.273824593
样本 16	28.26530612	20.67998982	-0.302131962
样本 17	28.81632653	21.38888564	-0.33070434
样本 18	29.36734694	22.10437018	-0.359528711
样本 19	29.91836735	22.82610395	-0.388592956
样本 20	30.46938776	23.55382012	-0.417885778
样本 21	31.02040816	24.28719574	-0.447396629
样本 22	31.57142857	25.02599764	-0.477115649
样本 23	32.12244898	25.76998741	-0.507033611
样本 24	32.67346939	26.5188887	-0.537141864
样本 25	33.2244898	27.27254579	-0.567432296
样本 26	33.7755102	28.03070185	-0.597897282
样本 27	34.32653061	28.7932104	-0.628529656
样本 28	34.87755102	29.55987541	-0.659322669
样本 29	35.42857143	29.9999601	-0.691153811
样本 30	35.97959184	29.9999601	-0.731774893
样本 31	36.53061224	29.9999601	-0.784276617
样本 32	37.08163265	29.9999601	-0.850009061
样本 33	37.63265306	29.9999601	-0.929950343
样本 34	38.18367347	29.9999601	-1.024627789
样本 35	38.73469388	29.9999601	-1.134076707
样本 36	39.28571429	29.9999601	-1.257838209

样本 37	39.83673469	29.9999601	-1.394992629
样本 38	40.3877551	29.9999601	-1.544221572
样本 39	40.93877551	29.9999601	-1.703889657
样本 40	41.48979592	29.9999601	-1.872136604
样本 41	42.04081633	29.9999601	-2.04697105
样本 42	42.59183673	29.9999601	-2.226359065
样本 43	43.14285714	29.9999601	-2.408302234
样本 44	43.69387755	29.9999601	-2.590902184
样本 45	44.24489796	29.9999601	-2.77241017
样本 46	44.79591837	29.9999601	-2.951261791
样本 47	45.34693878	29.9999601	-3.126097957
样本 48	45.89795918	29.9999601	-3.295773868
样本 49	46.44897959	29.9999601	-3.459358144
样本 50	47	29.9999601	-3.616124293

附录 12

介绍：问题三模型求解

BMI（区间起点）	最优孕周数
19.00	10.02
19.43	10.00
19.86	10.00
20.29	10
20.71	10
21.14	10.00
21.57	10.00
22.00	10
22.43	10.00
22.86	10.20
23.29	10.50
23.71	10.82
24.14	11.13
24.57	11.45
25.00	11.78
25.43	12.10
25.86	12.43
26.29	12.75
26.71	13.08
27.14	13.40

BMI（区间起点）	最优孕周数
27.57	13.73
28.00	14.05
28.43	14.36
28.86	14.67
29.29	14.97
29.71	15.27
30.14	15.54
30.57	15.81
31.00	16.04
31.43	16.24
31.86	16.39
32.29	16.46
32.71	16.33
33.14	14.85
33.57	15.59
34.00	16.34
34.43	17.08
34.86	17.83
35.29	18.57
35.71	19.32
36.14	20.06
36.57	20.81
37.00	21.55
37.43	22.30
37.86	23.04
38.29	23.79
38.71	24.53
39.14	25.28
39.57	26.02
40.00	26.77